

Backend Developer

—  
허준형

HEO JUNHYOUNG

☎ +82 10-6290-4016

✉ junhyoung.heo@gmail.com

소개

“ 동시성 제어, 데이터 정합성, 성능 최적화 를 통해  
안정적으로 동작하는 서비스를 설계하고 검증해 온 백엔드 엔지니어 ”



클라우드 인프라 자동화 및 운영

Terraform · AWS · GitOps



백엔드 성능 및 안정성

동시성 제어 · 데이터 정합성 · 성능 최적화



기록과 지식 공유

장애 분석 · 기술 문서화 · 300+ 포스팅



기술 스택

Backend

- Java
- Spring Boot
- JPA
- QueryDSL

Data & Messaging

- MySQL
- Redis
- MongoDB
- Kafka

Observability

- Prometheus
- Grafana
- k6

Infra & DevOps

- Docker · Kubernetes
- AWS
- Terraform
- GitHub Actions · ArgoCD



교육

아주대학교 사이버보안학과  
(2018.03 ~ 2024.08)

멋쟁이사자처럼 클라우드 엔지니어링  
(2025.03 ~ 2025.08)



자격증 & 어학

SQLD (2025.05)

OPIC IM3 (2024.07)



링크



GitHub



Blog



# 프로젝트 소개

개발자를 위한 포트폴리오 공유 및 커뮤니티 플랫폼

 개발 기간: 2025.10 ~ 2025.12

 팀 규모: 3 인

## 담당 역할

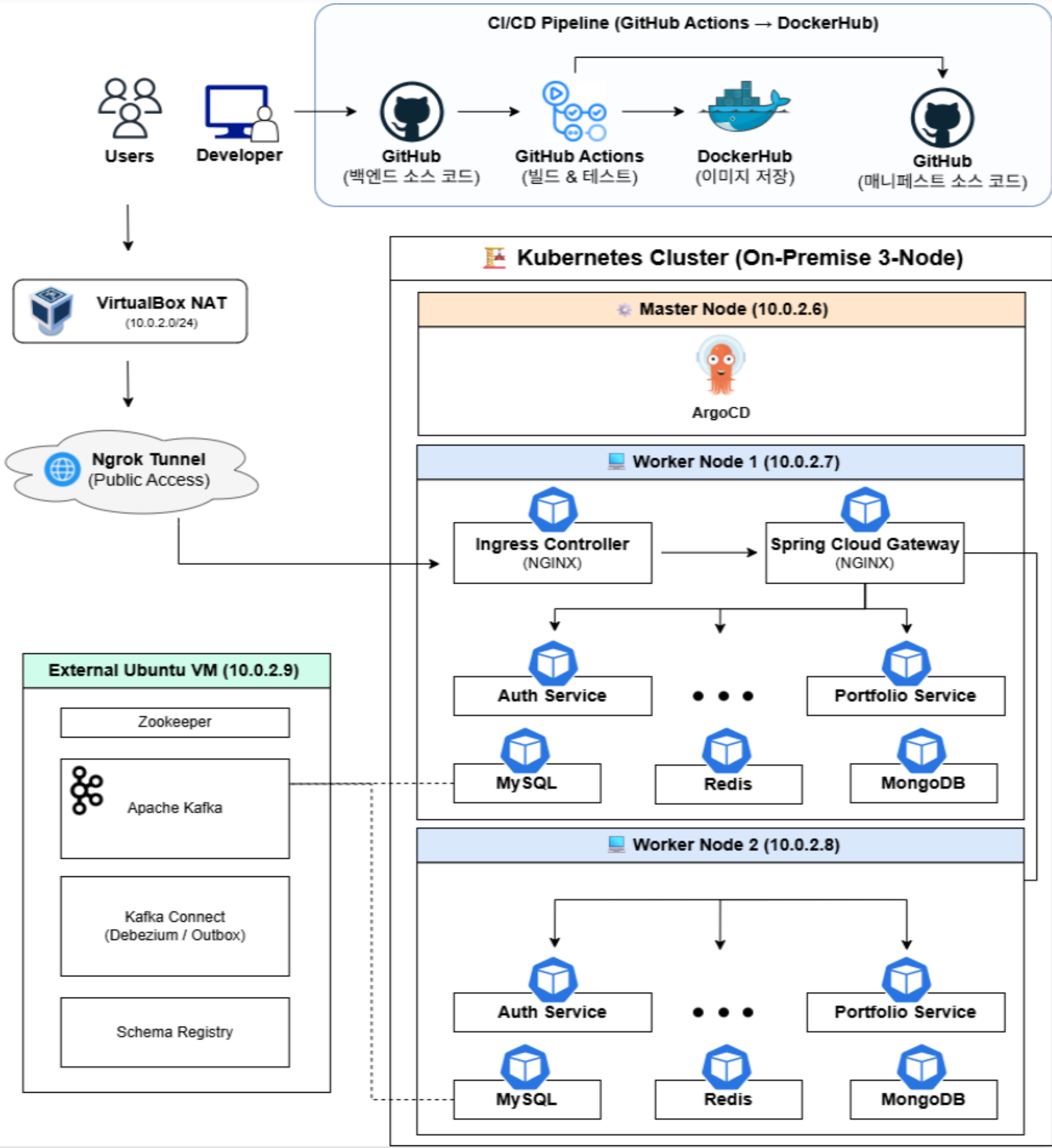
- ✓ MSA 기반 7개 서비스 설계 및 핵심 기능 구현
- ✓ 이벤트 기반 데이터 동기화(CDC) 및 Redis 캐싱 아키텍처 구축
- ✓ k6 기반 성능 테스트 및 병목 분석·최적화
- ✓ Kubernetes 기반 GitOps 배포 및 모니터링 환경 구축

 데이터 무결성 확보  
**99% +**  
CDC 최종 정합성 확보

 오류율  
**0%**  
k6 부하 테스트 기반 오류율 최소화

 조회 응답 속도  
**78%**  
Split Caching 도입 (p95)

 최대 TPS 처리량  
**2배**  
병목 해소 및 수용량 확장



# 아키텍처 의사결정

## 1 왜 Kafka + Debezium CDC 인가?

데이터 정합성을 보장하면서 서비스 간 결합도를 제거하기 위한 선택

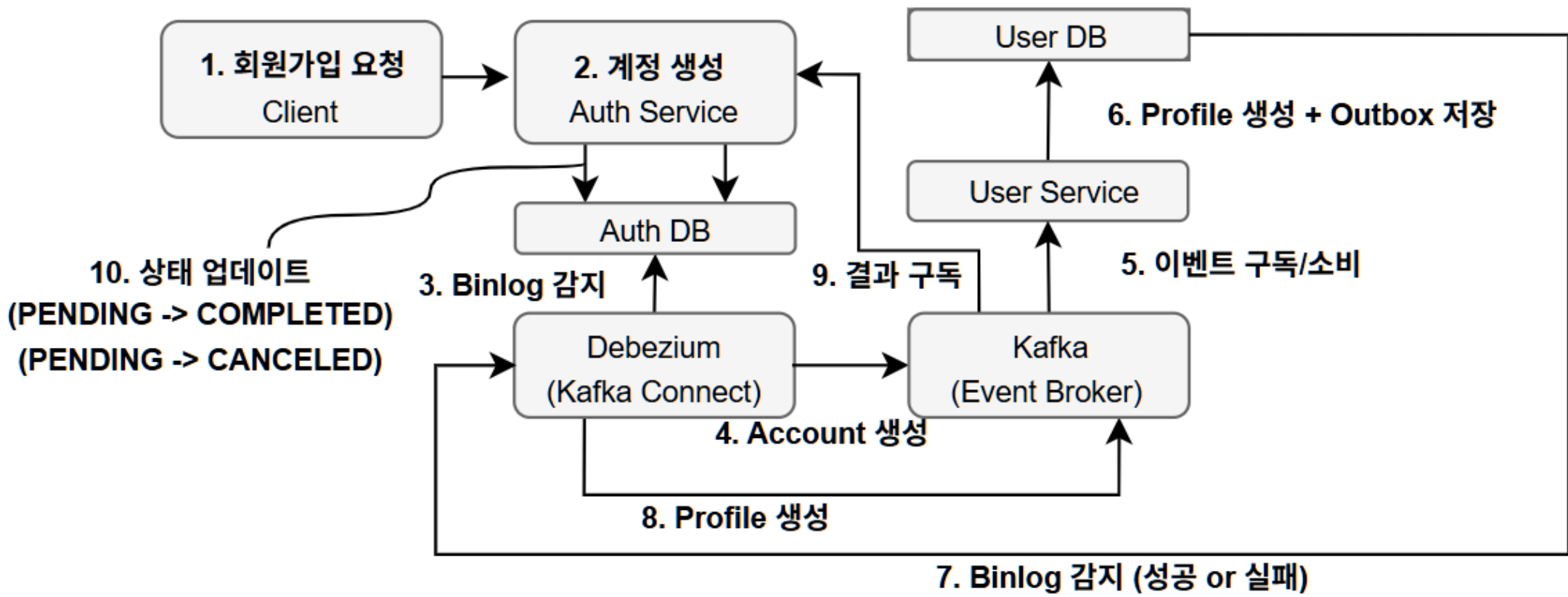
### 1. 배경 및 문제 상황

- Auth ↔ User 간 OpenFeign 동기 호출로 서비스 강결합
- 장애 전파 및 Dual Write로 데이터 정합성 위험

### 2. 대안 검토

|                                                                                                |               |
|------------------------------------------------------------------------------------------------|---------------|
| <div><div>✖</div><div><b>옵션 1. OpenFeign</b><br/>구현은 쉬우나 타 서비스 장애 시 회원가입 전체 마비</div></div>     | <div>기각</div> |
| <div><div>✖</div><div><b>옵션2. App 레벨 Event</b><br/>DB 트랜잭션과 메시지 발행의 원자성 보장 불가</div></div>      | <div>기각</div> |
| <div><div>✔</div><div><b>옵션 3. Outbox + Debezium</b><br/>DB 로그를 직접 캡처하여 데이터 정합성 보장</div></div> | <div>채택</div> |

### 3. 아키텍처 구현



### 4. 개선 결과

- ✔ 서비스 간 결합도 해소

✔ 장애 격리
- ✔ 데이터 정합성 보장

✔ 신뢰성 확보

2

왜 Redis Pub/Sub + MongoDB인가?

분산 환경에서 실시간 채팅의 인프라 확장성을 확보하고 N+1 네트워크 병목을 해결하기 위한 선택

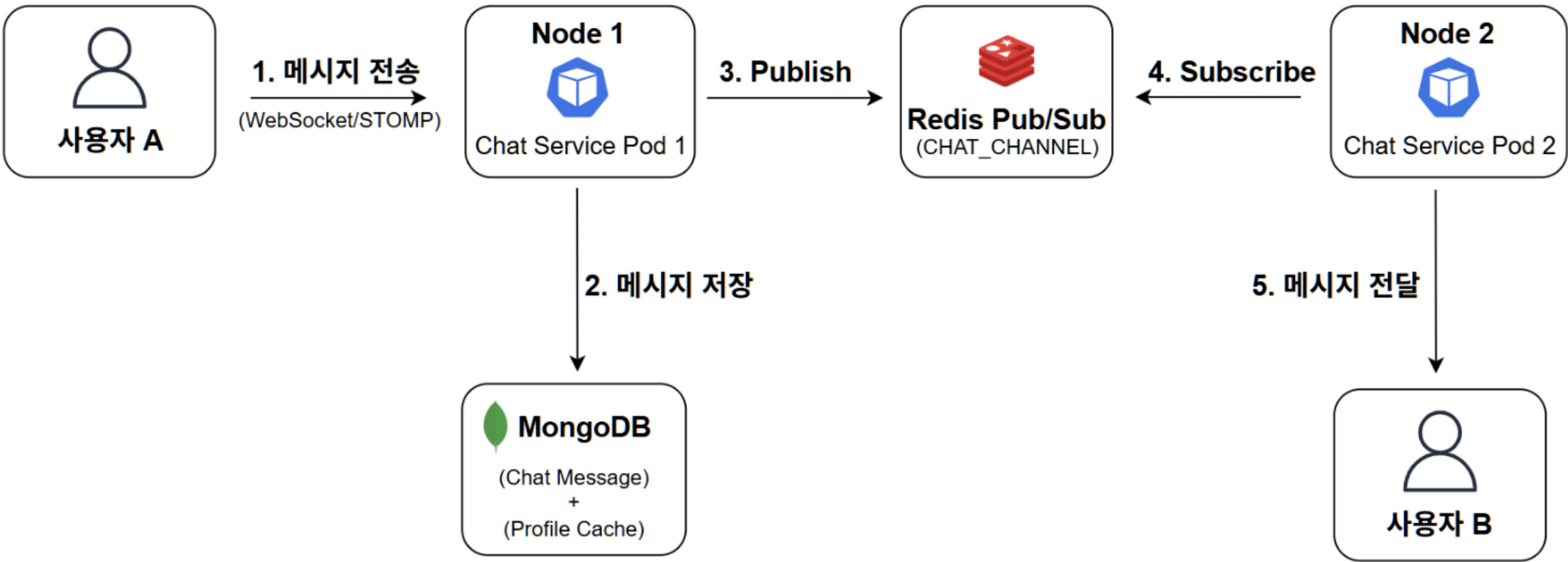
1. 배경 및 문제 상황

- Scale-Out 시 서버 간 메시지 단절
- User-Service 동기 호출로 N+1 발생

2. 대안 검토

|                                                                                                       |    |
|-------------------------------------------------------------------------------------------------------|----|
| <div><div>✖</div><div>옵션 1. OpenFeign</div><div>구현은 쉬우나 타 서비스 장애 시 회원가입 전체 마비</div></div>             | 기각 |
| <div><div>✖</div><div>옵션2. Kafka 확장</div><div>3-node on-premise 환경에서 JVM 자원 경합 및 운영 비용 증가</div></div> | 기각 |
| <div><div>✖</div><div>옵션3. RabbitMQ</div><div>낮은 숙련도에 따른 운영 리스크 존재</div></div>                        | 기각 |
| <div><div>✔</div><div>옵션 4. Redis Pub/Sub</div><div>초저지연 + 기존 Redis 활용</div></div>                    | 채택 |

3. 아키텍처 구현



4. 개선 결과

- ✔ Stateless 채팅 서버 구현

✔ User-Service API 호출 제거
- ✔ 분산 환경 메시지 전달

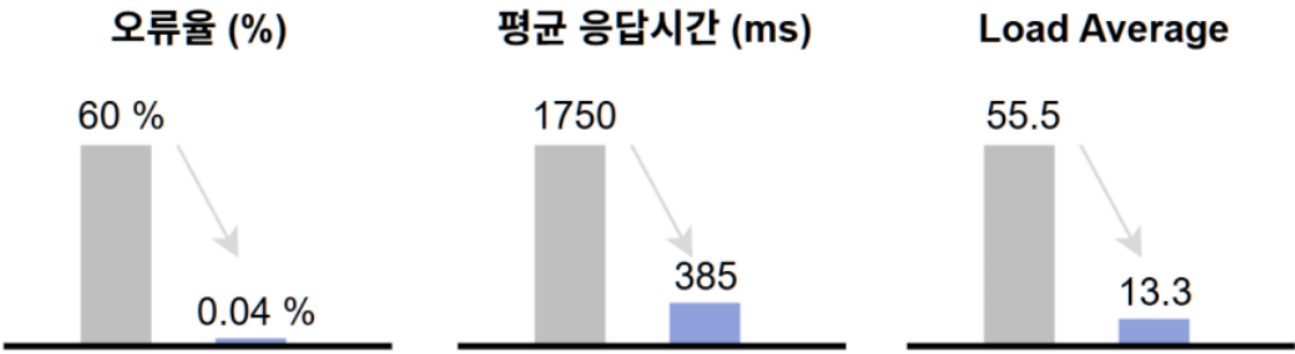
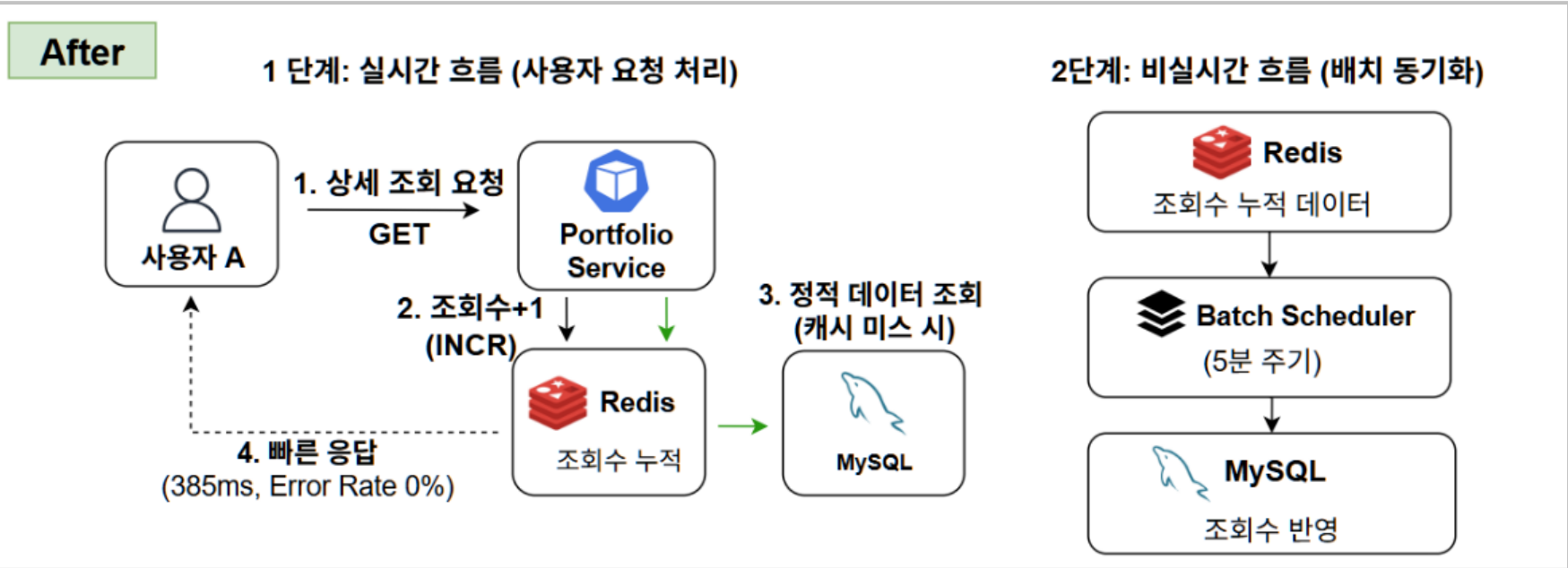
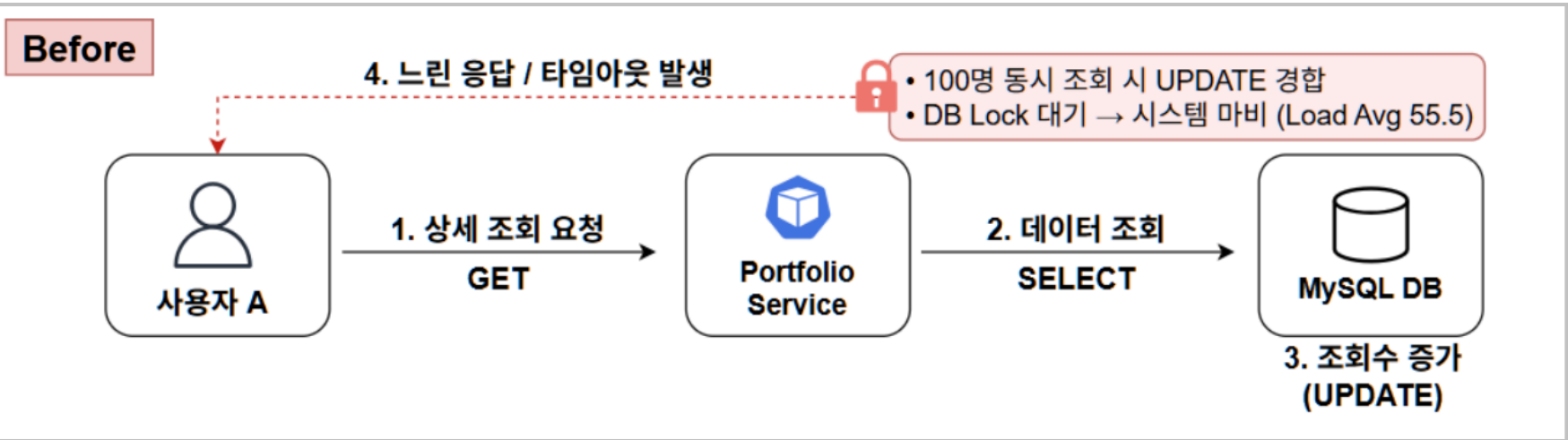
✔ N+1 문제 해결



# 성능 최적화

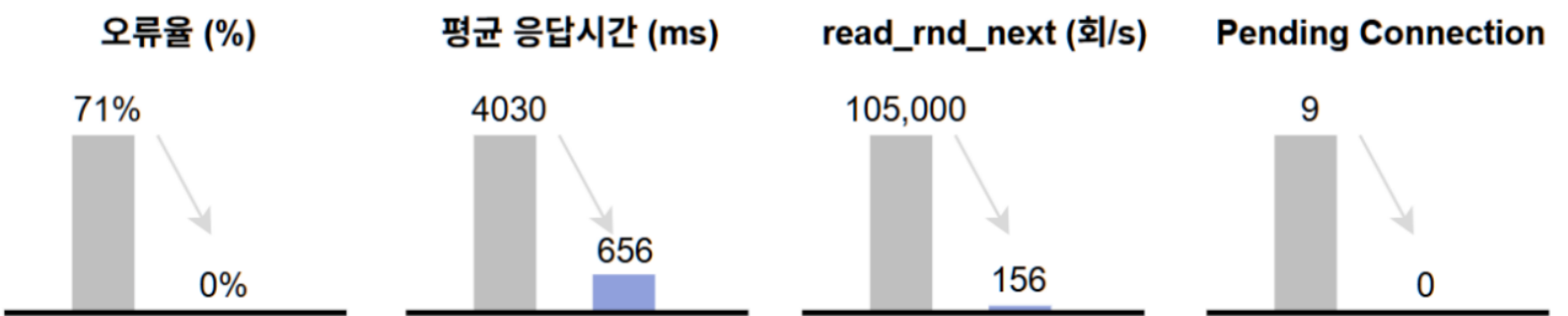
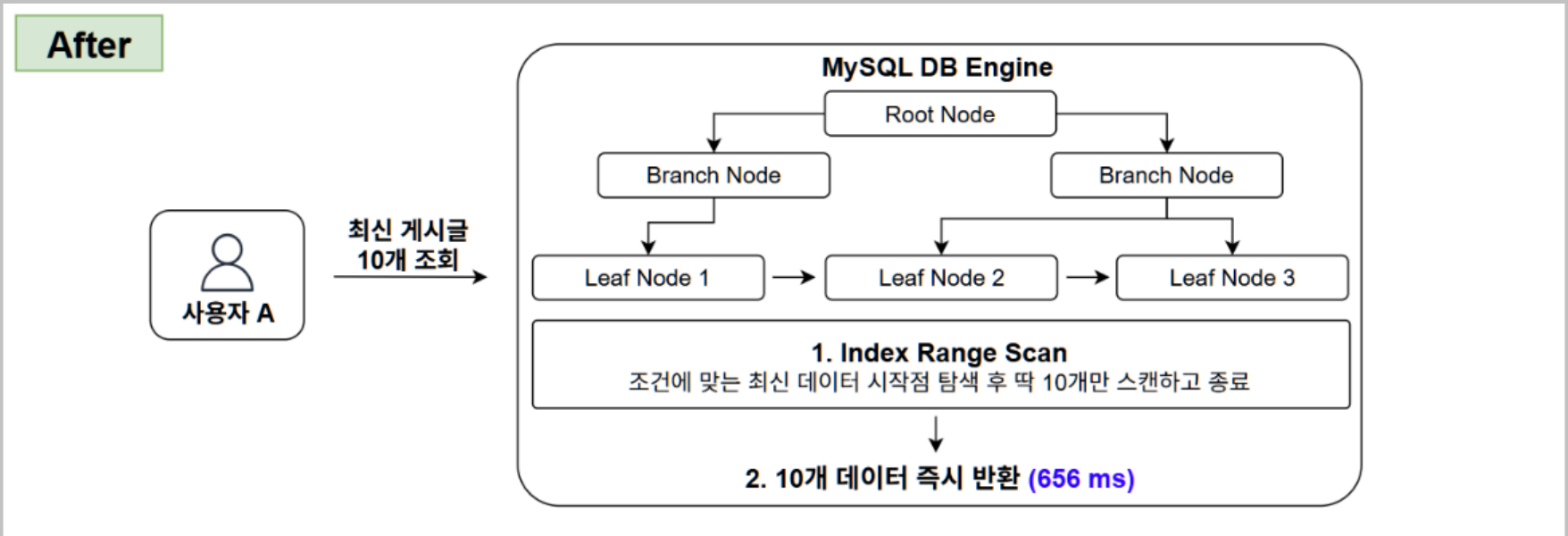
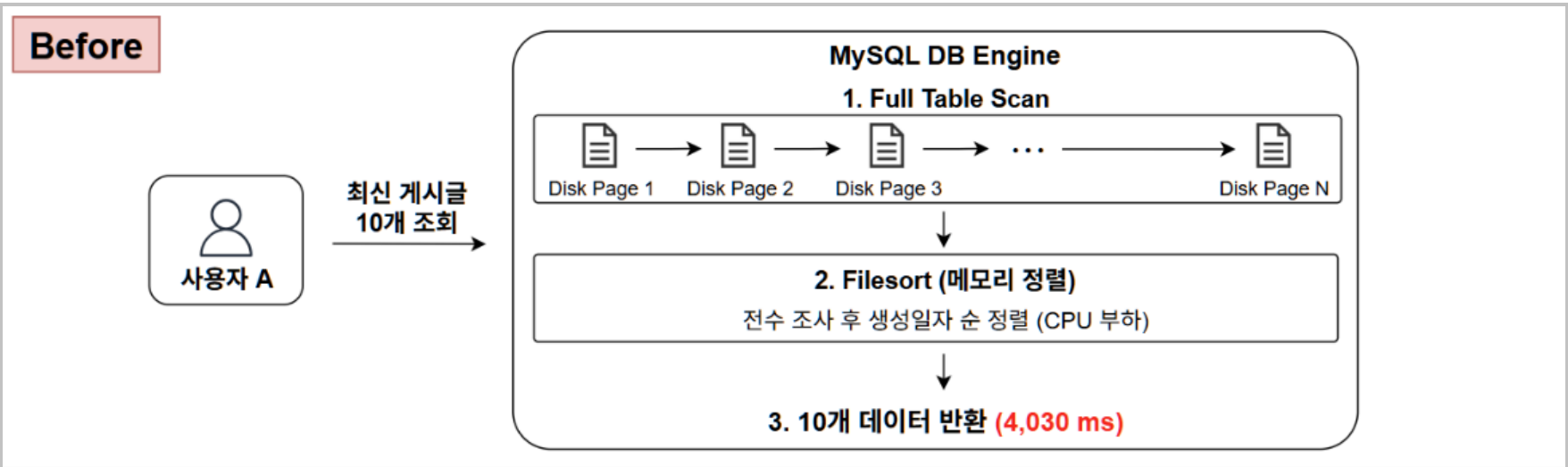
## 1 Split Cache 전략

조회수 증가 로직 분리를 통한 DB 부하 및 Lock 해소



## 2 Index Range Scan 최적화

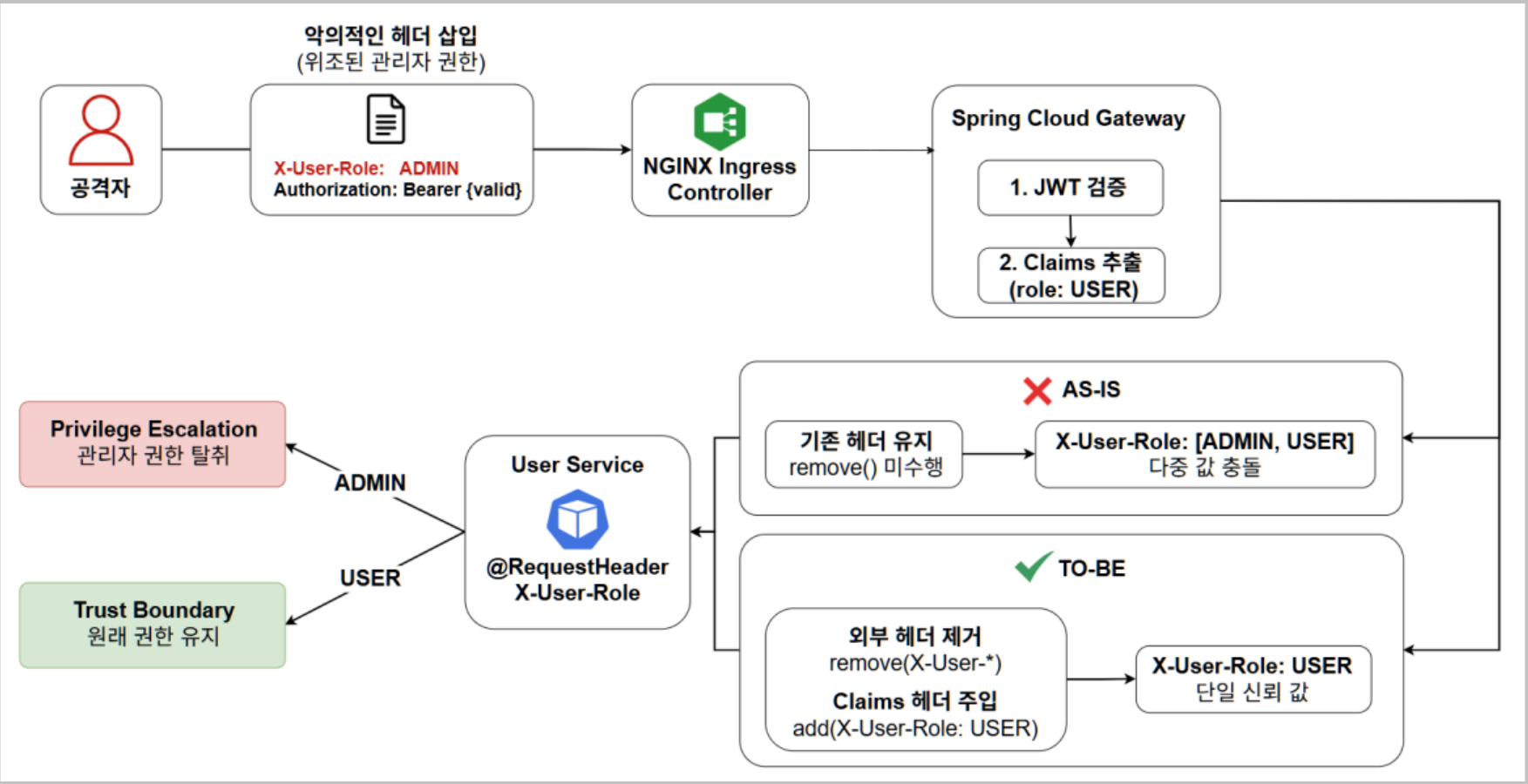
복합 인덱스 적용을 통한 Full Table Scan 제거





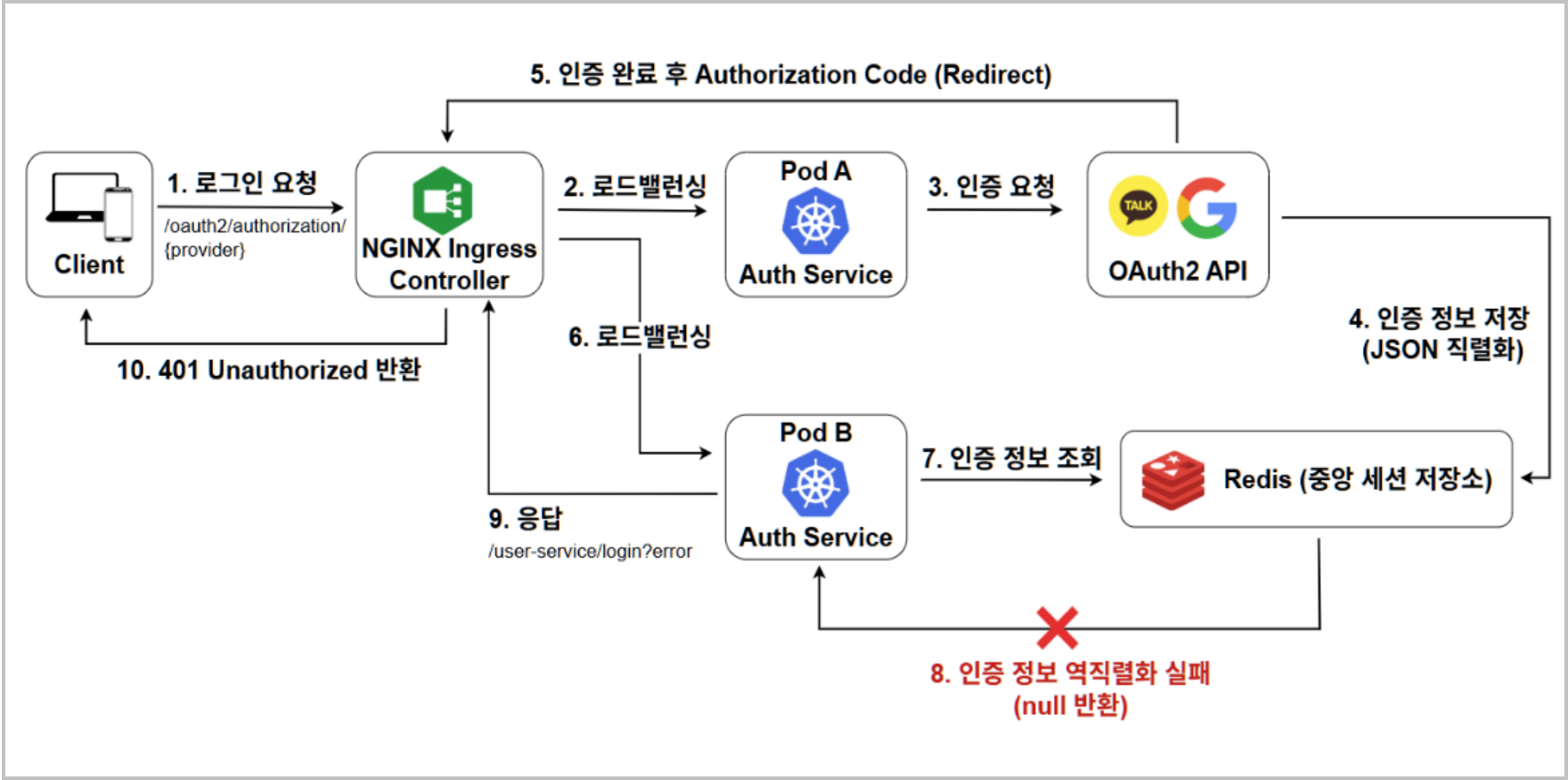
# 트러블 슈팅

## 1 Header Spoofing 취약점 분석 및 개선



- 상황 및 문제**
  - Authentication Offloading 구조 적용
  - 외부 X-User-\* 헤더 위조 가능성 발견
- 원인 분석**
  - Gateway가 기존 헤더를 제거하지 않음
  - 외부 헤더가 내부 서비스까지 전달
- 해결 과정**
  - X-User-\* 헤더 전체 제거
  - JWT Claims만 다시 주입하도록 변경
- 적용 결과**
  - Trust Boundary 확립
  - Header Spoofing 통한 권한 상승 차단

## 2 Multi-Pod 환경 OAuth2 로그인 401 Unauthorized 문제



- 상황 및 문제**
  - 다중 Pod 간 OAuth2 세션 공유를 위해 Redis 중앙 저장소 도입
  - 로드밸런싱 시 간헐적 401 발생 및 로그 없는 ?error 유실
- 원인 분석**
  - Pod A가 저장한 OAuth2 객체를 Redis에 JSON으로 저장
  - Pod B가 역직렬화 실패 후 null 반환 (CSRF 오인)
- 해결 과정**
  - JdkSerialization으로 직렬화 방식을 변경하여 객체 구조 보존
  - B 파드에서도 깨짐 없이 온전하게 OAuth2 State 데이터 복원
- 적용 결과**
  - Pod 간 데이터 해석 실패(null) 원천 차단으로 인증 일관성 확보
  - 스케일아웃 환경에서도 예외 없이 안정적인 소셜 로그인 구현

# 프로젝트 소개

신뢰를 기술로 증명하는 하이퍼 로컬(Hyper-local) 중고 거래 플랫폼

 개발 기간: 2026.01 ~ 2026.02

 팀 규모: 1 인

### 담당 역할

- 중고거래 비즈니스 로직 및 백엔드 API 개발
- JPA/JDBC 기반 데이터 계층 설계 및 대용량 벌크 insert 구현
- Redis 활용 캐싱 아키텍처 및 분산 락 동시성 제어 구축
- 비즈니스 매트릭 기반 모니터링 환경 구축



데이터 무결성 확보

99% +

낙관적·비관적 락 동시성 제어



DB 가용성 확보

96.7%

분산 락 기반 캐시 스템피드 차단



인프라 부하 절감

90%

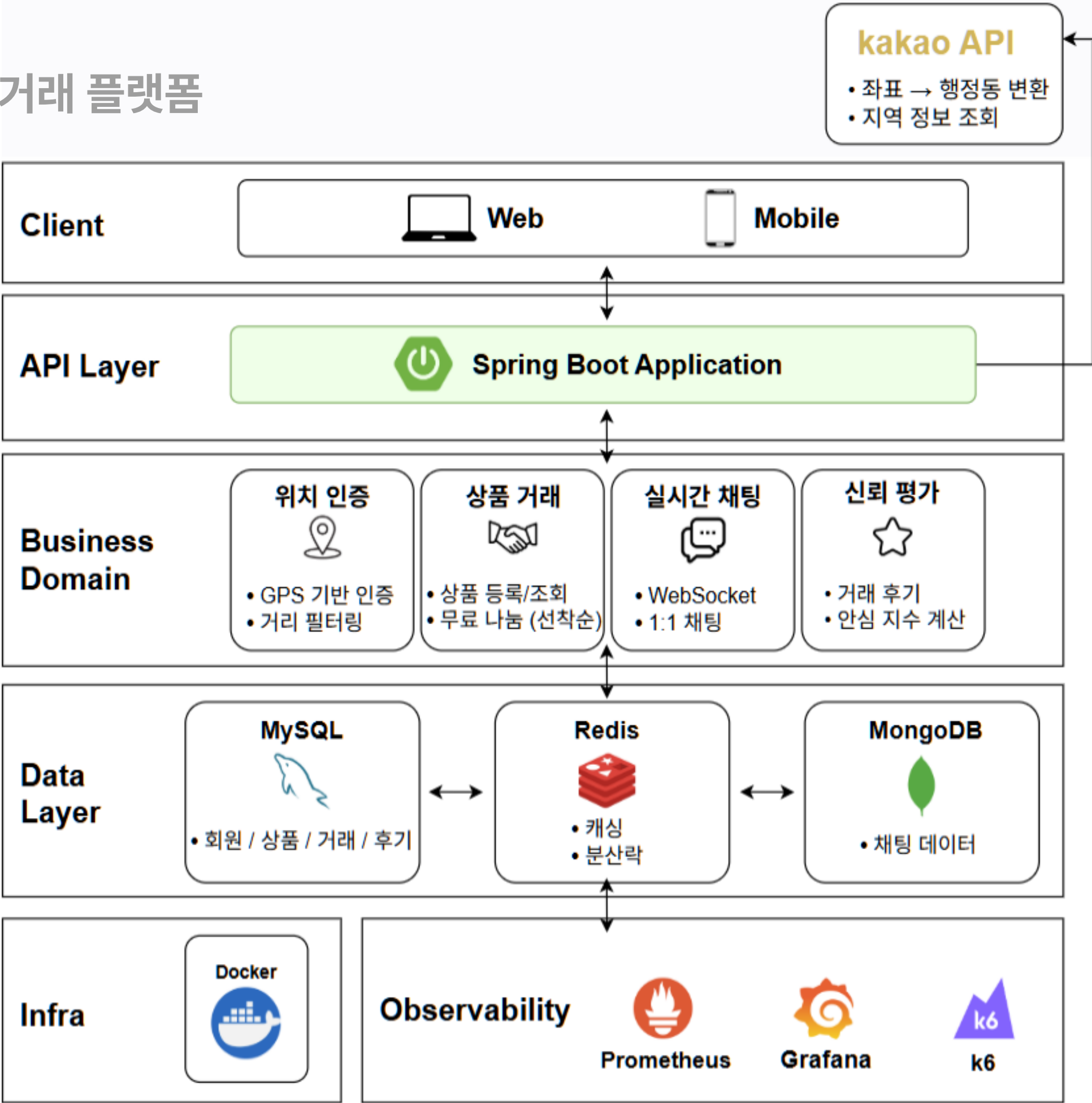
캐시 관통 방어 및 무효 요청 차단



검색 성능 향상

55%+

100만 데이터 기반 쿼리 최적화





# 아키텍처 의사결정

## 1 왜 Redis + Redisson + DCL 기반 캐싱 아키텍처를 선택했는가?

메인 페이지 조회 성능을 개선하면서 Cache Stampede로 인한 데이터베이스 부하를 방지하기 위한 선택

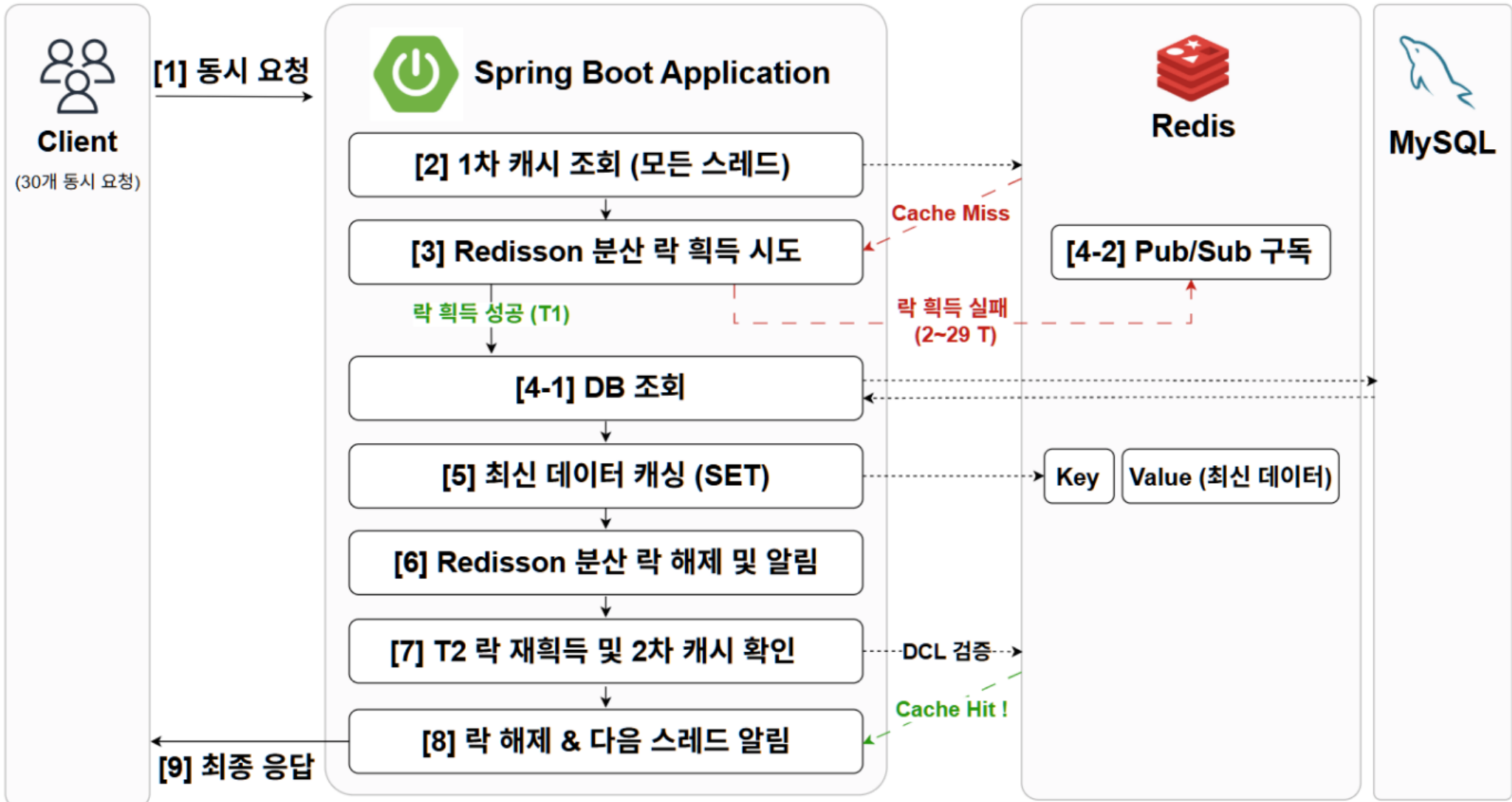
### 1. 배경 및 문제 상황

- 메인 페이지는 조회가 집중되는 Hot Data 영역
- 캐시 만료 시 동시 요청으로 DB 부하 증가 가능

### 2. 대안 검토

|                                                                                                           |    |
|-----------------------------------------------------------------------------------------------------------|----|
| <div><div>❌</div><div>옵션 1. DB 직접 조회</div><div>구현은 쉬우나 트래픽 증가 시 DB 부하 증가</div></div>                      | 기각 |
| <div><div>❌</div><div>옵션2. 단순 Redis Cache</div><div>조회 성능은 향상되지만,<br/>Cache Stampede 문제 발생 가능</div></div> | 기각 |
| <div><div>✅</div><div>옵션 3. Redis + Redisson + DCL</div><div>캐싱 문제점 해결 및 DB 부하 최소화</div></div>            | 채택 |

### 3. 아키텍처 구현



### 4. 개선 결과

☑ 응답 시간 개선

☑ Cache Stampede 방지

☑ DB 부하 감소

# 아키텍처 의사결정

## 2 왜 Atomic Update + UNIQUE 제약 기반 동시성 제어를 선택했는가?

한정된 재고의 동시성 경쟁을 해결하면서 데이터베이스 부하를 최소화하기 위한 선택

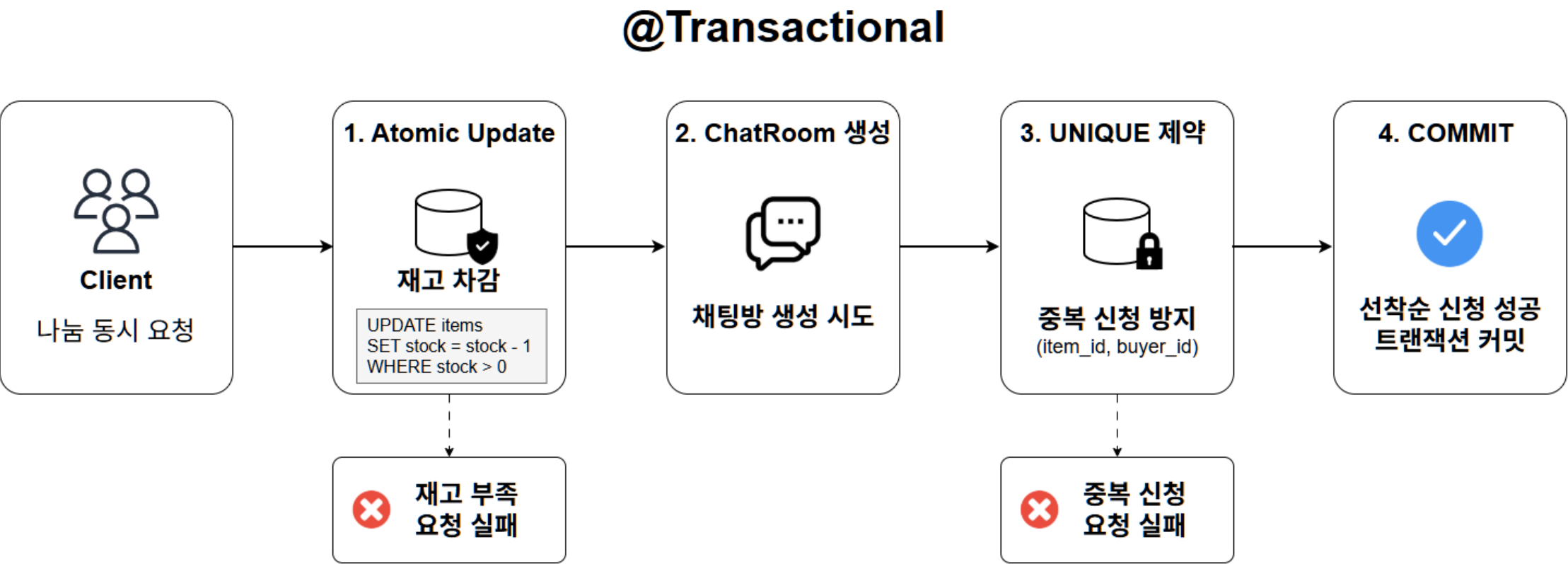
### 1. 배경 및 문제 상황

- 나눔 상품은 전형적인 동시성 경쟁 시나리오
- 채팅방 생성, 중복 신청 방지 등 하나의 트랜잭션으로 묶여있음

### 2. 대안 검토

|                                                                                                                      |    |
|----------------------------------------------------------------------------------------------------------------------|----|
| <div><div>✖</div><div>옵션 1. 비관적 락</div></div> <div>커넥션 풀 고갈 위험 및 채팅방 중복 생성 방지 불가</div>                               | 기각 |
| <div><div>✖</div><div>옵션2. Redis DECR</div></div> <div>메모리 기반 고성능 처리가 가능하지만,<br/>장애 대응 및 데이터 정합성 관리 필요</div>         | 기각 |
| <div><div>✔</div><div>옵션 3. 원자적 업데이트 + UNIQUE 제약조건</div></div> <div>DB 레벨에서 재고 검증과 중복 신청 해결 가능<br/>락 유지 시간 최소화</div> | 채택 |

### 3. 아키텍처 구현



### 4. 개선 결과

- ✔ DB 락 지속 시간 감소
- ✔ 재고 0 유지
- ✔ 커넥션 풀 부하 감소 및 처리량 향상

# 아키텍처 의사결정

## 3 왜 Elasticsearch 대신 MySQL FullText Index를 선택했는가?

현재 서비스 규모에 적합한 검색 성능을 확보하면서 운영 복잡도와 인프라 비용을 최소화하기 위한 선택

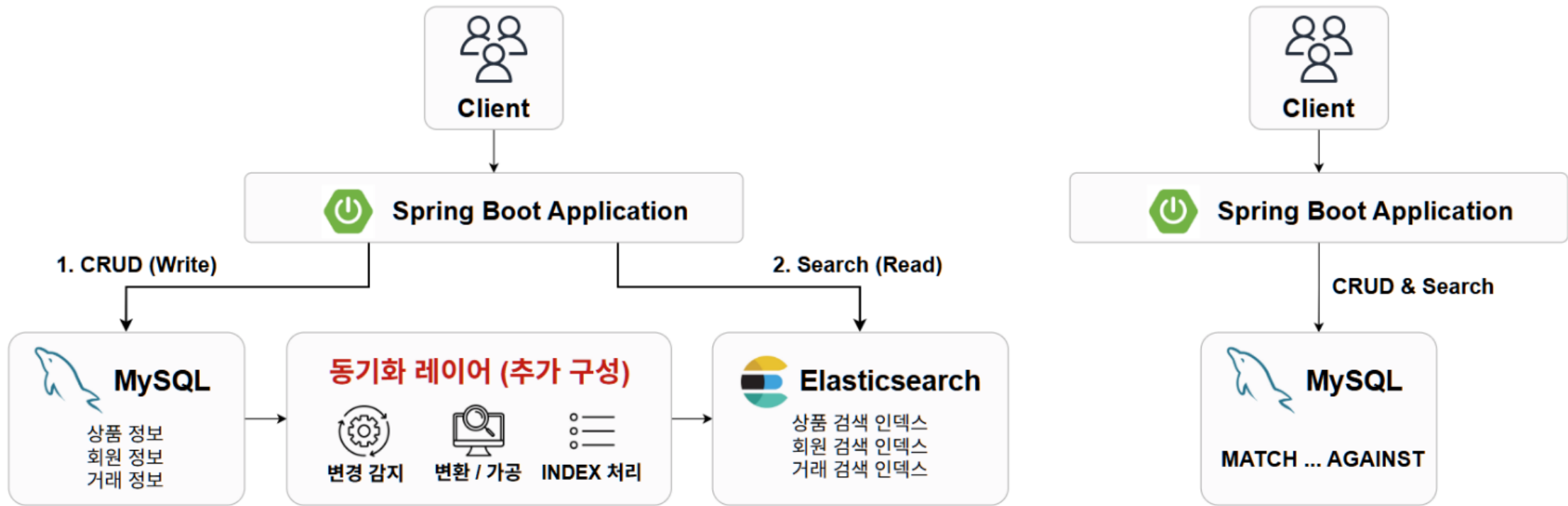
### 1. 배경 및 문제 상황

- 기존 LIKE 검색 사용 시 대량 데이터에서 Full Scan 발생
- 검색 응답 시간이 약 4.9초까지 증가

### 2. 대안 검토

|                                                                                                                 |    |
|-----------------------------------------------------------------------------------------------------------------|----|
| <div><div>✖</div><div>옵션 1. LIKE 검색</div><div>구현은 쉬우나 Full Table Scan 발생</div></div>                            | 기각 |
| <div><div>✖</div><div>옵션2. ElasticSearch</div><div>강력한 검색 기능 제공하지만,<br/>별도 클러스터 운영 및 데이터 동기화 필요</div></div>     | 기각 |
| <div><div>✔</div><div>옵션 3. MySQL FullText Index</div><div>기존 MySQL 인프라 활용 가능하여<br/>별도 검색 서버 운영 불필요</div></div> | 채택 |

### 3. 아키텍처 구현



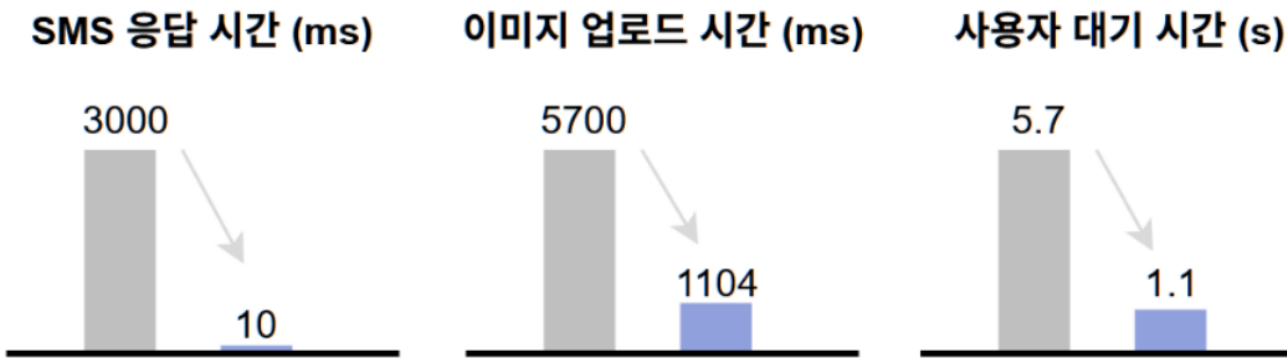
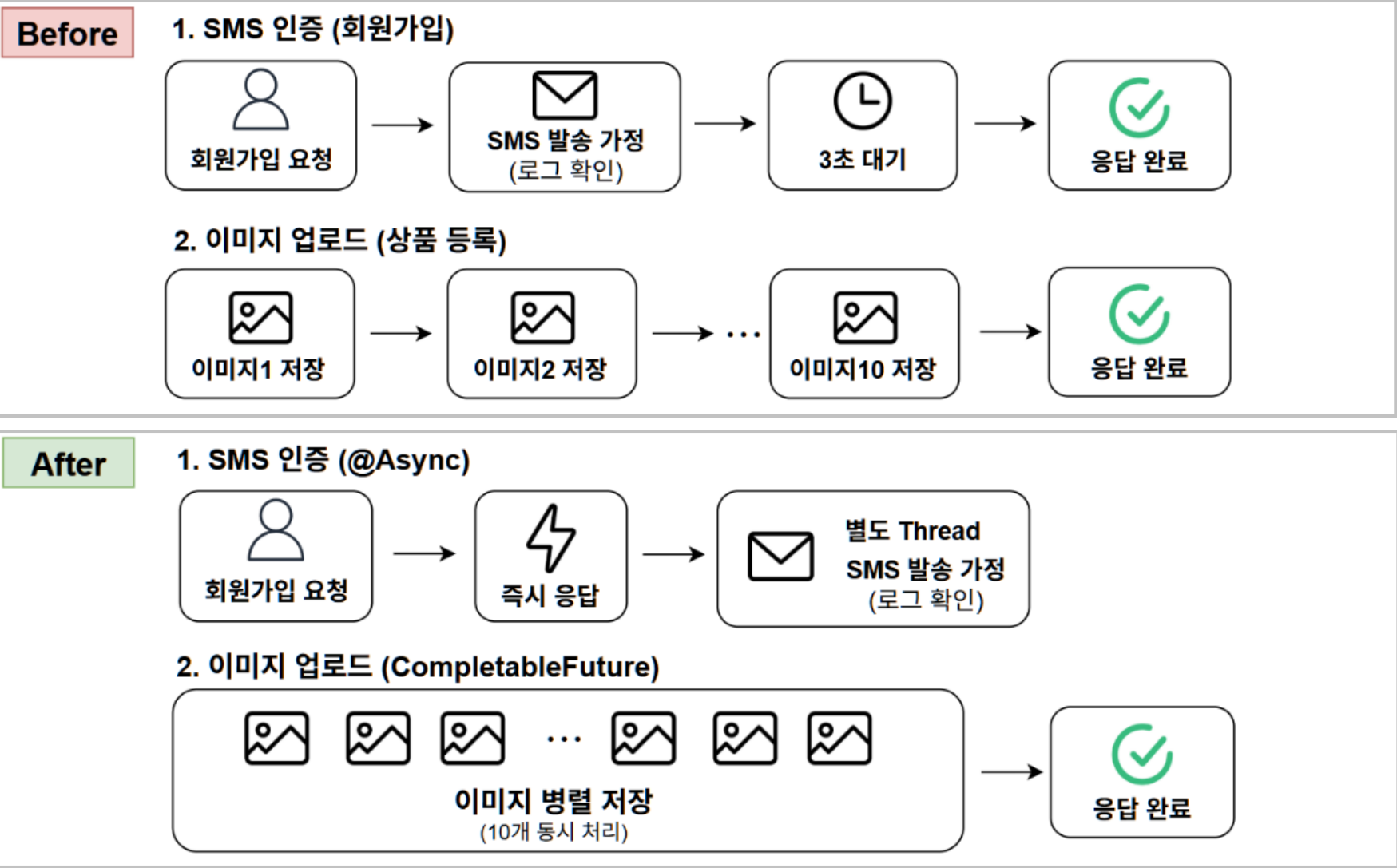
### 4. 개선 결과

- ✔ 검색 응답 시간 개선
- ✔ 별도 검색 엔진 불필요
- ✔ 운영 복잡도 및 인프라 비용 절감

# 성능 최적화

## 1 비동기 I/O 처리 최적화

@Async와 CompletableFuture를 활용한 응답 시간 개선



## 2 Query Count 기반 N+1 문제 최적화

Batch Fetch와 Fetch Join을 활용한 불필요한 DB 조회 제거

